RestaurantRush · Stage 1 · Class Diagram

organised by Boundary · Control · Entity

BOUNDARY · presentation & I/O
Main
+ main(args: String[]): void
+ play(in: InputStream, out: PrintStream, echo: boolean): void
«interface»
GameView
+ showTurnHeader(turn: int, maxTurns: int): void
+ showState(restaurant: Restaurant, stats: GameStats): void
+ chooseWaiterTask(tasks: List<WaiterTask>): WaiterTask
+ chooseCustomer(prompt: String, options: List<Customer>): Optional<Customer>
+ chooseOrder(prompt: String, options: List<Order>): Optional<Order>
+ chooseChefOrder(options: List<Order>): Optional<Order>
+ showMessage(message: String): void
+ showError(reason: String): void
+ showEndOfTurn(turn: int, maxTurns: int, stats: GameStats): void
+ showResult(result: GameResult): void
«creates»
view
1
ConsoleUI
- THICK: String
- THIN: String
- input: InputReader
- out: PrintStream
+ ConsoleUI(input: InputReader, out: PrintStream)
+ showTurnHeader(turn: int, maxTurns: int): void
+ showState(r: Restaurant, stats: GameStats): void
+ chooseWaiterTask(tasks: List<WaiterTask>): WaiterTask
+ chooseCustomer(prompt: String, options: List<Customer>): Optional<Customer>
+ chooseOrder(prompt: String, options: List<Order>): Optional<Order>
+ chooseChefOrder(options: List<Order>): Optional<Order>
- pickOrder(options: List<Order>, prompt: String): Optional<Order>
+ showMessage(message: String): void
+ showError(reason: String): void
+ showEndOfTurn(turn: int, maxTurns: int, stats: GameStats): void
+ showResult(result: GameResult): void
- describe(c: Customer): String
- describe(o: Order): String
- describeOrderStatus(o: Order): String
- progress(o: Order): String
- available(canAct: boolean, what: String): String
- mark(met: boolean): String
- formatAverage(avg: OptionalDouble): String
input
InputReader
- in: BufferedReader
- out: PrintStream
- echo: boolean
+ InputReader(in: InputStream, out: PrintStream, echo: boolean)
+ InputReader()
+ readInt(prompt: String, min: int, max: int): int
+ readYesNo(prompt: String): boolean
- readNextLine(prompt: String): String
«throws»
«static nested in InputReader»
InputEndedException
+ InputEndedException()
Money
- Money()
+ format(cents: int): String
CONTROL · game flow & staff
«enumeration»
WaiterTask
WAIT
SEAT
TAKE_ORDER
SERVE
- label: String
+ getLabel(): String
CustomerFactory
- nextId: int
+ create(type: CustomerType, turn: int): Customer
factory
«creates»
«uses»
GameController
- config: GameConfig
- restaurant: Restaurant
- stats: GameStats
- factory: CustomerFactory
- view: GameView
- turn: int
+ GameController(config: GameConfig, view: GameView)
+ run(): GameResult
- playTurn(): void
- handleArrivals(): void
- waiterChoice(): void
- performWaiterTask(task: WaiterTask): Optional<ActionResult>
- seatCustomer(): Optional<ActionResult>
- takeCustomerOrder(): Optional<ActionResult>
- serveOrder(): Optional<ActionResult>
- chefChoice(): void
- eatingAndPayment(): void
- waitingAndAbandonment(): void
- showActionResult(result: ActionResult): void
config
stats
restaurant
«abstract»
Staff
- name: String
- actedThisTurn: boolean
# Staff(name: String)
+ canAct(): boolean
# markActed(): void
+ resetTurn(): void
+ getName(): String
# busy(): ActionResult
# perform(action: Supplier<ActionResult>): ActionResult
«returns»
«final»
ActionResult
- ok: boolean
- message: String
- ActionResult(ok: boolean, message: String)
+ ok(message: String): ActionResult
+ fail(reason: String): ActionResult
+ isOk(): boolean
+ getMessage(): String
Waiter
- nextOrderId: int
+ Waiter()
+ seat(c: Customer, r: Restaurant): ActionResult
+ takeOrder(c: Customer, r: Restaurant): ActionResult
+ serve(o: Order, turn: int): ActionResult
waiter
Chef
+ Chef()
+ prepare(o: Order, turn: int): ActionResult
chef
Cashier
+ Cashier()
+ collect(due: List<Customer>, turn: int): Optional<Customer>
cashier
ENTITY · domain model
«create()»
arrivals
menu
Game state & session
GameConfig
- maxTurns: int
- tables: int
- targetPaid: int
- targetRevenueCents: int
- targetAvgSatisfaction: int
- arrivals: Map<Integer, CustomerType>
- menu: Menu
+ GameConfig(maxTurns: int, tables: int, targetPaid: int, targetRevenueCents: int, targetAvgSatisfaction: int, arrivals: Map<Integer, CustomerType>, menu: Menu)
+ baseSetting(): GameConfig
+ arrivalAt(turn: int): Optional<CustomerType>
+ getMaxTurns(): int
+ getTables(): int
+ getTargetPaid(): int
+ getTargetRevenueCents(): int
+ getTargetAvgSatisfaction(): int
+ getMenu(): Menu
config
GameResult
- victory: boolean
- paidMet: boolean
- revenueMet: boolean
- satisfactionMet: boolean
- stats: GameStats
- config: GameConfig
~ GameResult(victory: boolean, paidMet: boolean, revenueMet: boolean, satisfactionMet: boolean, stats: GameStats, config: GameConfig)
+ isVictory(): boolean
+ isPaidMet(): boolean
+ isRevenueMet(): boolean
+ isSatisfactionMet(): boolean
+ getStats(): GameStats
+ getConfig(): GameConfig
GameStats
- revenueCents: int
- paidSatisfactions: List<Integer>
- servedCount: int
- unhappyDepartures: int
- turnsUsed: int
+ recordPayment(cents: int, satisfaction: int): void
+ recordServed(): void
+ recordDeparture(): void
+ recordTurnPlayed(turn: int): void
+ averageSatisfaction(): OptionalDouble
+ evaluate(c: GameConfig): GameResult
+ getRevenueCents(): int
+ getPaidCount(): int
+ getServedCount(): int
+ getUnhappyDepartures(): int
+ getTurnsUsed(): int
stats
«creates»
Composition owns tables, queue & staff. Encapsulation seat() is package-private; staff must go through it.
Restaurant
- tables: List<Table>
- queue: WaitingQueue
- menu: Menu
- customers: List<Customer>
- waiter: Waiter
- chef: Chef
- cashier: Cashier
+ Restaurant(tableCount: int, menu: Menu)
+ resetStaff(): void
+ admit(c: Customer): void
+ freeTable(): Optional<Table>
+ quoteBill(c: Customer, item: MenuItem): int
~ seat(c: Customer, byWhom: String): ActionResult
+ awaitingOrder(): List<Customer>
+ activeOrders(): List<Order>
+ kitchenOrders(): List<Order>
+ readyToPay(): List<Customer>
+ presentCustomers(): List<Customer>
+ finishEating(turn: int): List<Customer>
+ collectPayment(turn: int): Optional<Customer>
+ applyWaiting(): List<Customer>
- customersIn(state: CustomerState): List<Customer>
- orders(): Stream<Order>
+ getTables(): List<Table>
+ getQueue(): WaitingQueue
+ getMenu(): Menu
+ getCustomers(): List<Customer>
+ getWaiter(): Waiter
+ getChef(): Chef
+ getCashier(): Cashier
tables
queue
customers
Customer subsystem
«enumeration»
CustomerType
REGULAR
- label: String
- constructor: BiFunction<Integer, Integer, Customer>
+ getLabel(): String
+ create(id: int, arrivalTurn: int): Customer
«instantiates»
Table
- id: int
- occupant: Customer
+ Table(id: int)
+ getId(): int
+ isFree(): boolean
+ getOccupant(): Customer
~ seat(c: Customer): void
~ release(): void
occupant / table
«abstract»
Customer
- id: int
- arrivalTurn: int
- satisfaction: int
- state: CustomerState
- table: Table
- order: Order
- servedTurn: int
- readyToPayTurn: int
+ Customer(id: int, arrivalTurn: int)
+ waitPenalty(): int
+ getType(): CustomerType
+ computeBill(baseCents: int): int
+ chooseItem(m: Menu): MenuItem
+ isWaitingForService(): boolean
+ isPresent(): boolean
+ loseSatisfaction(): void
+ hasGivenUp(): boolean
+ finishEating(currentTurn: int): boolean
+ canPayOn(currentTurn: int): boolean
~ seatAt(t: Table): void
~ placeOrder(o: Order): void
~ onServed(turn: int): void
~ pay(): void
~ leave(): void
+ getId(): int
+ getArrivalTurn(): int
+ getSatisfaction(): int
+ getState(): CustomerState
+ getTable(): Table
+ getOrder(): Order
+ getServedTurn(): int
+ getReadyToPayTurn(): int
+ getName(): String
WaitingQueue
- customers: Deque<Customer>
+ enqueue(c: Customer): void
+ remove(c: Customer): boolean
+ isEmpty(): boolean
+ size(): int
+ snapshot(): List<Customer>
customers
state
«enumeration»
CustomerState
WAITING
SEATED
ORDERED
SERVED
READY_TO_PAY
PAID
LEFT
Polymorphism abstract waitPenalty() and getType() are implemented by RegularCustomer; GameController never checks the type.
RegularCustomer
+ RegularCustomer(id: int, arrivalTurn: int)
+ waitPenalty(): int
+ getType(): CustomerType
order / customer
Menu & orders
Order
- id: int
- customer: Customer
- item: MenuItem
- billCents: int
- unitsDone: int
- status: OrderStatus
- readyTurn: int
+ Order(id: int, customer: Customer, item: MenuItem, billCents: int)
+ getId(): int
+ getCustomer(): Customer
+ getItem(): MenuItem
+ getBillCents(): int
+ getUnitsDone(): int
+ getStatus(): OrderStatus
+ getReadyTurn(): int
+ isInKitchen(): boolean
+ isReady(): boolean
+ isActive(): boolean
~ advance(turn: int): void
~ markServed(): void
~ markPaid(): void
~ cancel(): void
+ describe(): String
status
«enumeration»
OrderStatus
PLACED
IN_PREPARATION
READY
SERVED
PAID
CANCELLED
item
MenuItem
- name: String
- priceCents: int
- prepUnits: int
+ MenuItem(name: String, priceCents: int, prepUnits: int)
+ getName(): String
+ getPriceCents(): int
+ getPrepUnits(): int
items
«final»
Menu
- items: List<MenuItem>
+ Menu(items: List<MenuItem>)
+ standard(): Menu
+ size(): int
+ get(index: int): MenuItem
+ getItems(): List<MenuItem>
1
1..*
0..1
0..*
«creates»
menu
Legend
Architectural layer (lanes)
Boundary – CLI views, input, formatting
Control – turn flow, staff actions
Entity – domain state & rules
Stage introduced (class fill)
Stage 1 – base game
Design note (OO principle)
Relationships
—▷ inheritance
- - ▷ realization
◆— composition
◇— aggregation
—> association
- - > dependency
underline static
italic abstract